



中山大學  
SUN YAT-SEN UNIVERSITY

# 本科生实验报告

实验课程: 操作系统原理实验

任课教师: 刘宁

实验题目: 中断

专业名称: 计算机科学与技术

学生姓名: 林宏宇

学生学号: 23320093

实验地点: 实验中心 D501

实验时间: 2025 年 4 月 9 日

## Section 1 实验概述

在本章中，我们首先介绍一份 C 代码是如何通过预编译、编译、汇编和链接生成最终的可执行文件。接着，为了更加有条理地管理我们操作系统的代码，我们提出了一种 C/C++ 项目管理方案。在做了上面的准备工作后，我们开始介绍 C 和汇编混合编程方法，即如何在 C 代码中调用汇编代码编写的函数和如何在汇编代码中调用使用 C 编写的函数。介绍完混合编程后，我们来到了本章的主体内容——中断。我们介绍了保护模式下的中断处理机制和可编程中断部件 8259A 芯片。最后，我们通过编写实时钟中断处理函数来将本章的所有内容串联起来。

通过本章的学习，同学们将掌握使用 C 语言来编写内核的方法，理解保护模式的中断处理机制和处理时钟中断，为后面的二级分页机制和多线程/进程打下基础。

## Section 2 预备知识与实验环境

该节总结实验需要用到的基本知识，以及主机型号、代码编译工具、重要三方库的版本号信息等。

- 预备知识：x86 汇编语言程序设计、Linux 系统命令行工具
- 实验环境：
  - 虚拟机版本/处理器型号：Virtualbox
  - 代码编辑环境：Vscode+nasm+C/C++ 插件
  - 代码编译工具：gcc/g++（64 位）
  - 重要三方库信息：

## Section 3 实验任务

- 实验任务 1：完成混合编程的基本思路
- 实验任务 2：完成使用 C/C++ 编写内核
- 实验任务 3：完成中断的处理
- 实验任务 4：完成时钟中断的处理

## Section 4 实验步骤与实验结果

### ----- 实验任务 1 -----

- 任务要求：混合编程的基本思路。

复现网址中“一个混合编程的例子”部分。

要求：

1. 将原例子中最后一行的输出“Done”（参考下图）改为“Done by 学号 姓名首字母”
2. 结合具体的代码说明 C 代码调用汇编函数的语法和汇编代码调用 C 函数的语法。例如，结合关键代码说明 `global`、`extern` 关键字的作用，为什么 C++ 的函数前需要加上 `extern "C"` 等，保存结果截图并说说你是怎么做的；
3. 学习 `make` 的使用，并用 `make` 来构建项目，保存结果截图并说说你是怎么做。

● 思路分析：

我们首先需要通过 C、C++ 和汇编语言实现函数的调用，并且确保能成功编译并运行。在此过程中，重点包括：

1. 如何在 C、C++ 和汇编语言之间正确传递参数；
2. 使用 `global` 和 `extern` 关键字来确保跨语言调用的正确性；
3. 在 C++ 中使用 `extern "C"` 来避免 C++ 的函数重载问题；
4. 通过 `Makefile` 来自动化整个构建过程。

● 实验步骤：

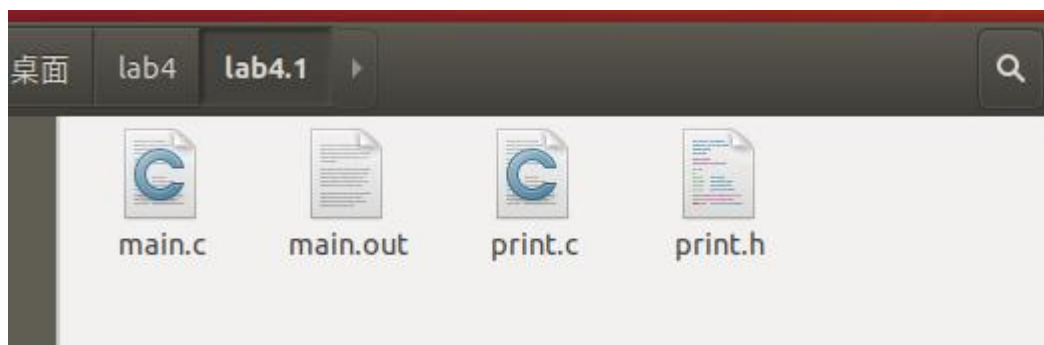
1. 从代码到可执行文件

1.1 代码结构

再 `print.h` 的头文件，声明一个名为 `print_something()` 的函数

在 `print.c` 中实现这个函数

在 `main.c` 中使用这个函数打印并输出



1.2 编译

编译 `main.c`，然后生成可执行文件 `main.out`。其中，`-o` 指定了生成的可执行文件的名称。hello.c 不知道函数 `print_something` 的实现。而函数 `print_something` 的实现在文件 `print.c` 中，我们需要在编译命令中加上它。

```
gcc -o main.out main.c print.c
```

1.3 运行

```
./main.out
```

2. 代码编译的四个阶段

2.1 预处理

处理宏定义，如 `#include`，`#define`，`#ifndef` 等，生成预处理文件，一般以 `.i` 为后缀。

```
gcc -o main.i -E main.c
```

2.2 编译

将预处理文件转换成汇编代码文件，一般以.s为后缀。

说明手册上此处代码有误，见终端输出

```
gcc -o main.s -S main.c -masm=intel
```

### 2.3 汇编

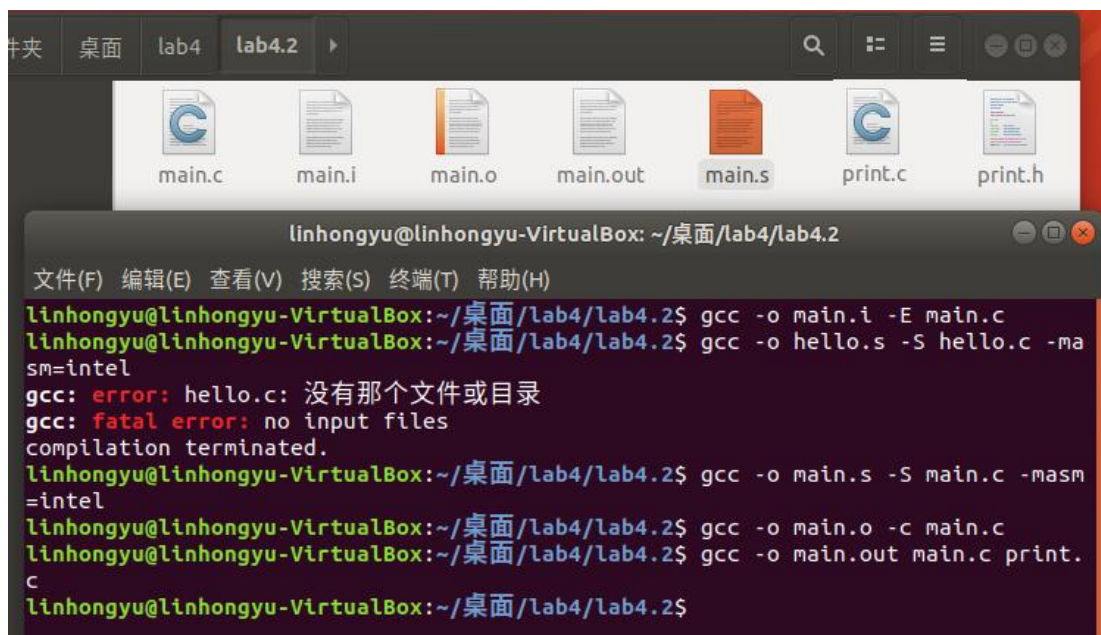
将汇编代码文件转换成可重定位文件，一般以.o为后缀。

```
gcc -o main.o -c main.c
```

### 2.4 链接

将多个可重定位文件链接生成可执行文件，一般以.out为后缀。

```
gcc -o main.out main.c print.c
```



## 3. 复现案例，输出自己的“Done by 学号 姓名首字母”

### 3.1 代码结构

在文件 c\_func.c 中定义 C 函数 function\_from\_C。

在文件 cpp\_func.cpp 中定义 C++ 函数 function\_from\_CPP。

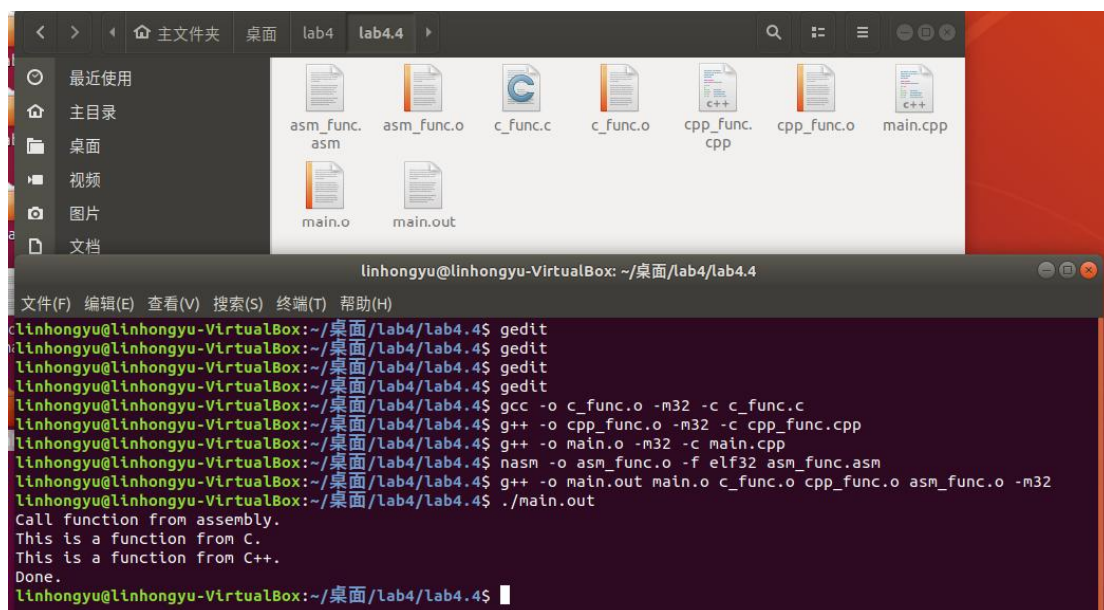
在文件 asm\_func.asm 中定义汇编函数 function\_from\_asm，在 function\_from\_asm 中调用 function\_from\_C 和 function\_from\_CPP。

在文件 main.cpp 中调用汇编函数 function\_from\_asm

### 3.2 编译

```
gcc -o c_func.o -m32 -c c_func.c
g++ -o cpp_func.o -m32 -c cpp_func.cpp
g++ -o main.o -m32 -c main.cpp
nasm -o asm_func.o -f elf32 asm_func.asm
g++ -o main.out main.o c_func.o cpp_func.o asm_func.o -m32
```

代码结构和编译结果如图



### 3.3 修改函数，使得输出为“Done by 学号 姓名首字母”

```
int main() {  
    std::cout << "Call function from assembly." << std::endl;  
    function_from_asm();  
    std::cout << "Done by 23320093LHY." << std::endl;  
}
```

4. 结合具体的代码说明 C 代码调用汇编函数的语法和汇编代码调用 C 函数的语法。说明 global、extern 关键字的作用为什么 C++ 的函数前需要加上 extern “C”。

#### 4.1 C 代码调用汇编函数

调用汇编函数之前，我们先需要在汇编代码中将函数声明为 global，否则在链接阶段会找不到函数的实现。

```
global function_from_asm
```

然后我们在 C/C++ 中将其声明来自外部即可，如下所示。

```
extern void function_from_asm();
```

#### 4.2 C++ 代码调用汇编函数

在 C++ 中需要声明为 extern “C”，如下所示。

```
extern "C" void function_from_asm();
```

#### 4.3 汇编代码调用 C 函数

需要在汇编代码中声明这个函数来自于外部。

```
extern function_from_C
```

声明后便可直接使用，例如。

```
call function_from_C
```

#### 4.4 汇编代码调用 C++ 函数

需要在 C++ 代码的函数定义前加上 extern “C”。因为 C++ 支持函数重载，为了区别同名的重载函数，C++ 在编译时会进行名字修饰。也就是说，

function\_from\_CPP 编译后的标号不再是 function\_from\_CPP，而是要带上额外的信息。而 C 代码编译后的标号还是原来的函数名。因此，extern "C" 目的是告诉编译器按 C 代码的规则编译，不进行名字修饰。

```
extern "C" void function_from_CPP();
```

同样地，在汇编代码中声明这个函数即可。

```
extern function_from_CPP
```

汇编代码调用 C 函数过程

```
int function_from_C(int arg1, int arg2);
```

当我们需要在汇编代码中调用函数 function\_from\_C，调用的形式是 function\_from\_C(1,2)，此时的汇编代码如下。

```
push 2      ; arg2
push 1      ; arg1
call function_from_C
add esp, 8   ; 清除栈上的参数
```

#### 4.5 C 函数调用汇编代码

```
extern int function_from_asm(int arg1, int arg2);
```

当我们需要在 C 代码中调用函数 function\_from\_asm 时，使用如下语句即可。

```
int ret = function_from_asm(1, 2);
```

函数调用后，参数 2，1，返回值被依次放在栈上

### 5. 学习 make 的使用，并用 make 来构建项目

#### 5.1 原理

编译程序的过程如下。

将.c、.cpp，.asm 编译成目录文件.o。

使用链接器将.o 文件合成一个文件。

因此，上面的例子的 Makefile 如下。

```
main.out: main.o print.o
    gcc -o main.out main.o print.o
print.o: print.c
    gcc -o print.o -c print.c
main.o: main.c
    gcc -o main.o -c main.c
clean:
    rm *.o *.out
```

然后编译运行。

```
make
./main.out
```

#### 5.2 使用 make 创建项目

使用一个简单的项目来展示 makefile 的使用。

编写 makefile 代码：

```
# 编译器和目标文件
CC = gcc
```

```

TARGET = my_program

SRC = main.c mathlib.c

# 默认规则
$(TARGET): $(SRC)

    $(CC) $(SRC) -o $(TARGET)

# 运行规则
run: $(TARGET)

    ./$(TARGET)

# 清理规则
clean:

    rm -f $(TARGET)

```

使用指令，可见结果输出

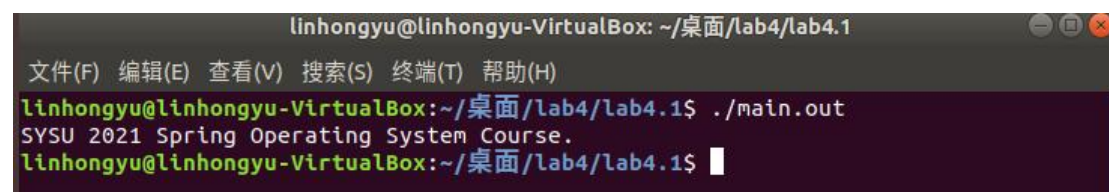
```

make
make run

```

## ● 实验结果展示：

### 1. 从代码到可执行文件

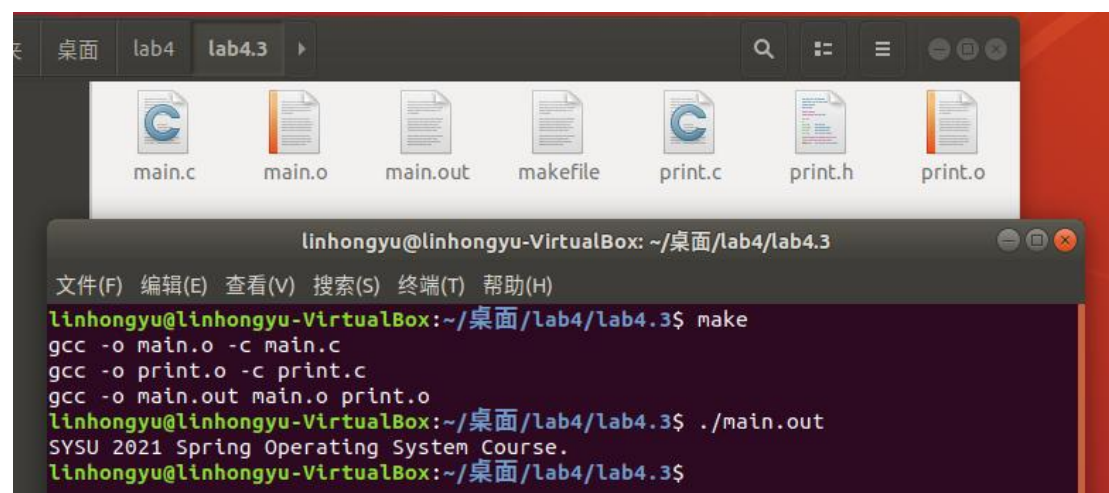


```

linhongyu@linhongyu-VirtualBox: ~/桌面/lab4/lab4.1
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.1$ ./main.out
SYSU 2021 Spring Operating System Course.
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.1$

```

### 2. 四个阶段



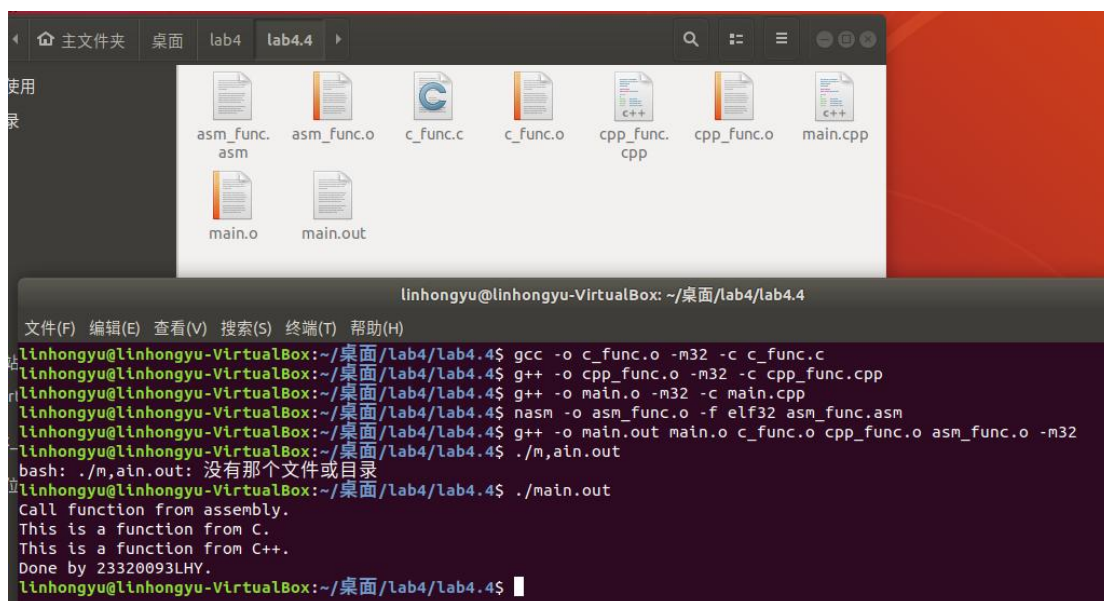
```

linhongyu@linhongyu-VirtualBox: ~/桌面/lab4/lab4.3
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.3$ make
gcc -o main.o -c main.c
gcc -o print.o -c print.c
gcc -o main.out main.o print.o
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.3$ ./main.out
SYSU 2021 Spring Operating System Course.
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.3$

```

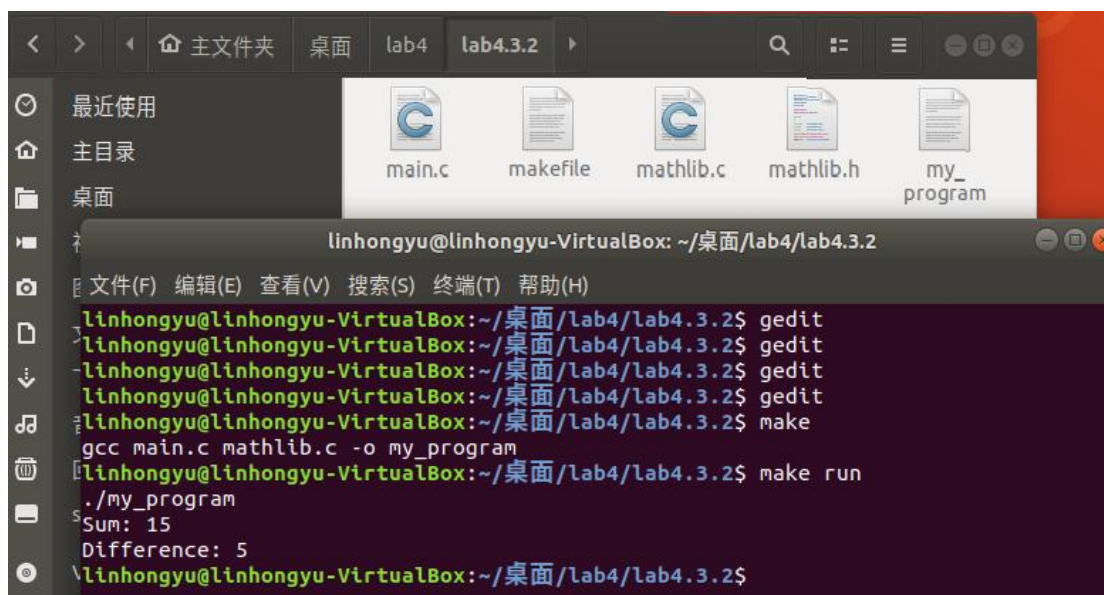
### 3. 混合编程，输出自己的学号和姓名





```
linhongyu@linhongyu-VirtualBox: ~/桌面/lab4/lab4.4
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.4$ gcc -o c_func.o -m32 -c c_func.c
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.4$ g++ -o cpp_func.o -m32 -c cpp_func.cpp
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.4$ g++ -o main.o -m32 -c main.cpp
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.4$ nasm -o asm_func.o -f elf32 asm_func.asm
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.4$ g++ -o main.out main.o c_func.o cpp_func.o asm_func.o -m32
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.4$ ./main.out
bash: ./main.out: 没有那个文件或目录
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.4$ ./main.out
Call function from assembly.
This is a function from C.
This is a function from C++.
Done by 23320093LHY.
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.4$
```

#### 4. 使用 makefile 的例子



```
linhongyu@linhongyu-VirtualBox: ~/桌面/lab4/lab4.3.2
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.3.2$ gedit
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.3.2$ gedit
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.3.2$ gedit
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.3.2$ gedit
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.3.2$ make
gcc main.c mathlib.c -o my_program
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.3.2$ make run
./my_program
Sum: 15
Difference: 5
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.3.2$
```

### ----- 实验任务 2 -----

- 任务要求：完成使用 C/C++编写内核

复现网址中“内核的加载”部分，在进入 `setup_kernel` 函数后，将输出 `Hello World` 改为输出“学号+姓名首字母”，保存结果截图并说说你是怎么做的。

- 思路分析：主要分析如何得到预期结果，有哪些要注意的地方等。

内核加载的过程：

在 `setup_kernel` 函数中，通常会有一些初始化操作（例如打印“Hello World”）。我们需要找到这一部分代码并修改。

内核代码一般使用汇编语言进行输出，因为内核通常在没有标准输出设备（如显示器和键盘）的情况下直接操作硬件，因此可能会使用特定的端口进行字符显示。



汇编代码的修改：

输出内容可能是通过修改内存映射的显存来实现的。原始代码中输出的是“Hello World”，我们需要将其修改为学号和姓名的首字母。

修改代码时的注意事项：

需要确保修改的字符编码是正确的。通常，在内核中，我们使用 16 位的字符模式来显示文本，每个字符占两个字节。

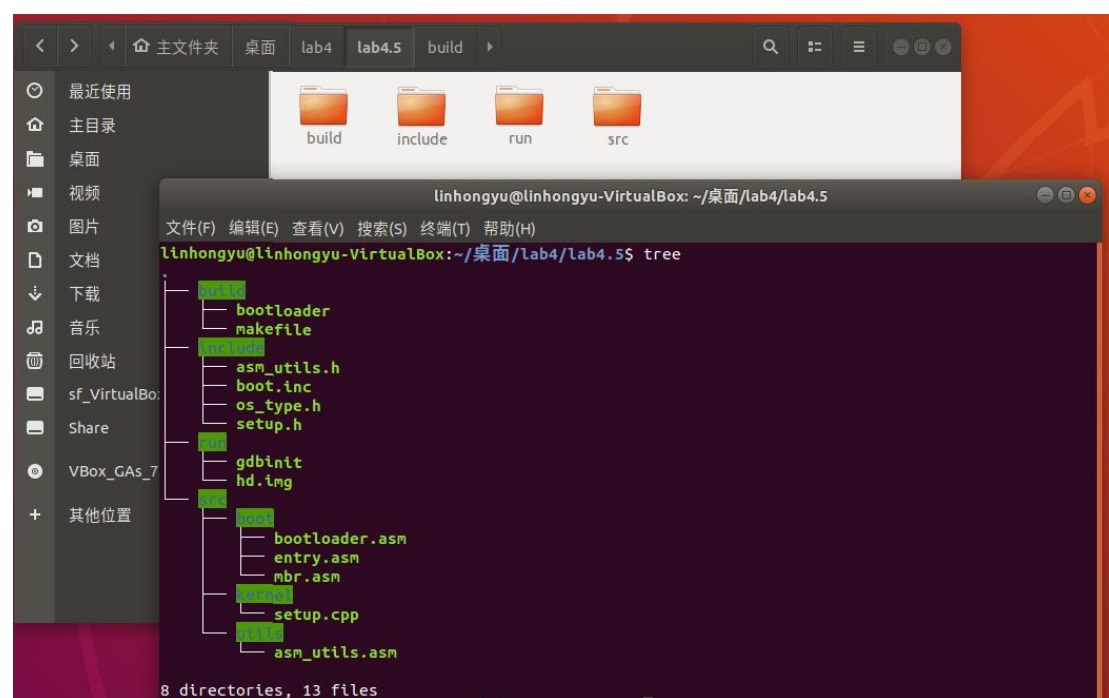
需要确认字符串输出的内存位置是否正确，确保写入正确的地址。

## ● 实验步骤：

### 1. 展示代码结构

下载 tree

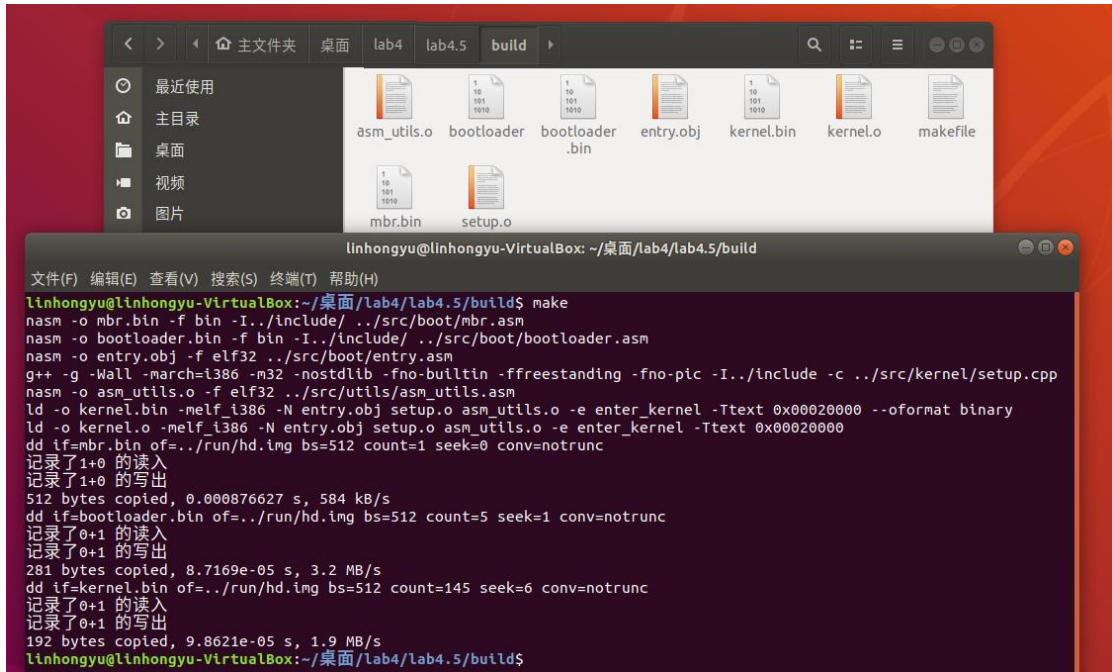
```
sudo snap install tree
```



### 2. 编译

使用 makefile 进行编译，提高效率

```
make
```



### 3. 运行 复现完成

```
make run
```

### 4. 修改代码 以输出自己的学号

```
[bits 32]

global asm_hello_world
asm_hello_world:

    push eax

    xor eax, eax

    mov ah, 0x03 ;青色

    mov al, '2'

    mov [gs:2 * 0], ax

    mov al, '3'

    mov [gs:2 * 1], ax

    mov al, '3'

    mov [gs:2 * 2], ax

    mov al, '2'

    mov [gs:2 * 3], ax

    mov al, '0'

    mov [gs:2 * 4], ax

    mov al, '0'

    mov [gs:2 * 5], ax

    mov al, '9'

    mov [gs:2 * 6], ax

    mov al, '3'

    mov [gs:2 * 7], ax
```

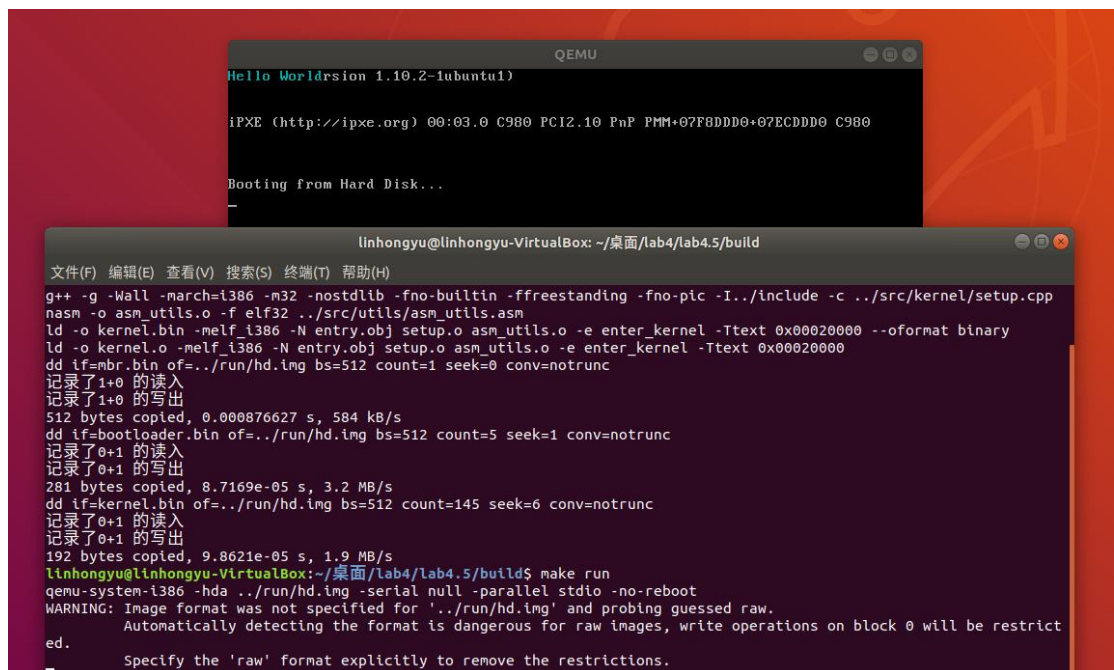
```
pop eax
Ret
```

重新编译运行

```
Make
Make run
```

- 实验结果展示：通过执行前述代码，可得下图结果。

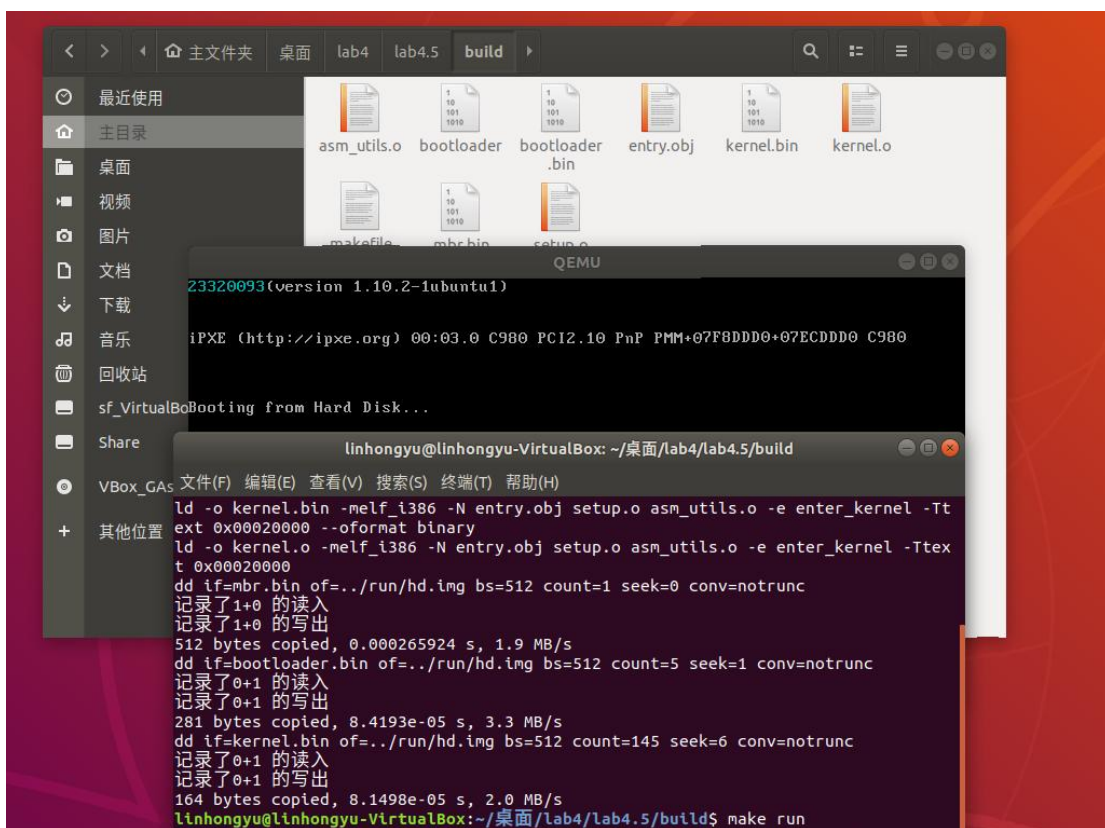
## 1. 复现结果



The screenshot shows two windows. The top window is a QEMU virtual machine titled 'QEMU' with a black background and white text. It displays 'Hello Worldrsion 1.10.2-1ubuntu1)' and 'iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DD0+07ECDD0 C980'. Below this, it says 'Booting from Hard Disk...' and a cursor is visible. The bottom window is a terminal titled 'linhongyu@linhongyu-VirtualBox: ~/桌面/lab4/lab4.5/build'. It shows the compilation of a kernel using g++ and nasm, followed by the execution of 'make run'. The terminal output includes the compilation commands and the execution of 'qemu-system-i386' with various options. It also shows the progress of writing data to a hard disk image, including the size of the data and the time taken.

```
linhongyu@linhongyu-VirtualBox: ~/桌面/lab4/lab4.5/build
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
g++ -g -Wall -march=i386 -m32 -nostdlib -fno-builtin -ffreestanding -fno-pic -I../include -c ../src/kernel/setup.cpp
nasm -o asm_utils.o -f elf32 ../src/utls/asm_utils.asm
ld -o kernel.bin -melf_i386 -N entry.obj setup.o asm_utils.o -e enter_kernel -Ttext 0x00020000 --oformat binary
ld -o kernel.o -melf_i386 -N entry.obj setup.o asm_utils.o -e enter_kernel -Ttext 0x00020000
dd if=mbr.bin of=../run/hd.img bs=512 count=1 seek=0 conv=notrunc
记录了1+0 的读入
记录了1+0 的写出
512 bytes copied, 0.000876627 s, 584 kB/s
dd if=bootloader.bin of=../run/hd.img bs=512 count=5 seek=1 conv=notrunc
记录了0+1 的读入
记录了0+1 的写出
281 bytes copied, 0.7169e-05 s, 3.2 MB/s
dd if=kernel.bin of=../run/hd.img bs=512 count=145 seek=6 conv=notrunc
记录了0+1 的读入
记录了0+1 的写出
192 bytes copied, 9.8621e-05 s, 1.9 MB/s
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.5/build$ make run
qemu-system-i386 -hda ../run/hd.img -serial null -parallel stdio -no-reboot
WARNING: Image format was not specified for '../run/hd.img' and probing guessed raw.
Automatically detecting the format is dangerous for raw images, write operations on block 0 will be restrict
ed.
Specify the 'raw' format explicitly to remove the restrictions.
```

## 2. 输出学号



### ----- 实验任务 3 -----

- 任务要求：中断的处理。

复现网址中“初始化 IDT”部分，你可以更改默认的中断处理函数为你编写的函数，然后触发之，对结果进行截图并说说你是怎么做的。要求：调用处理函数时输出个人学号或姓名信息。

- 思路分析：

IDT（中断描述符表）的作用：

IDT 是中断和异常处理的核心数据结构，它包含了每个中断向量号和对应的中断处理程序地址。在程序启动时需要初始化 IDT 并加载。

每当发生中断时，CPU 会查找 IDT，找到相应的处理程序并跳转执行。

中断处理函数的设计：

中断处理函数通常会包括两部分：一是保存现场（例如使用 push 和 pop 指令保存寄存器值），二是进行实际的中断处理（例如输出信息）。

由于我们需要输出学号或姓名信息，在处理函数中会涉及到字符输出操作。通常，在内核编程中，输出是通过操作显存或使用特定的端口进行的。

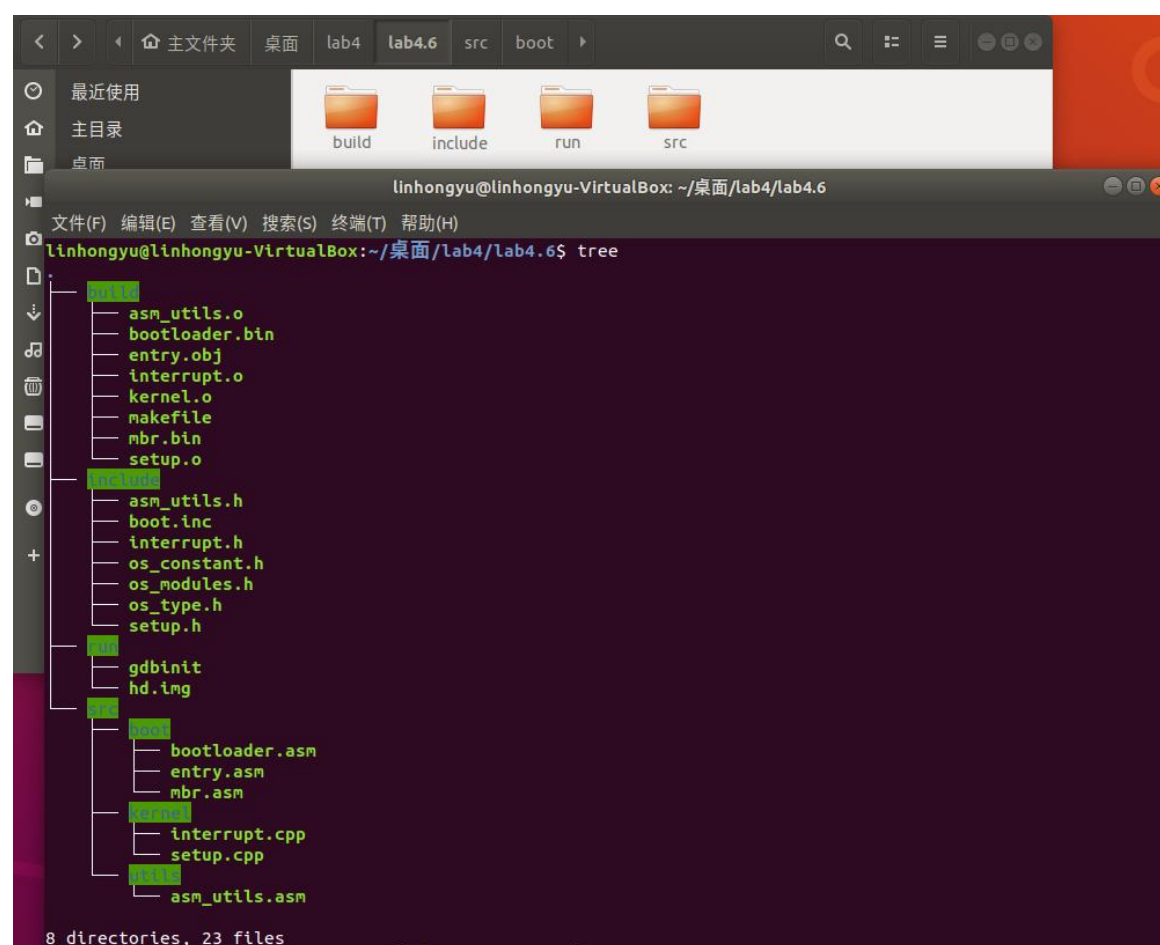
如何输出学号或姓名信息：

我们将在汇编代码中定义一个字符串（例如 `ASM_UNHANDLED_INTERRUPT_INFO`），用于存储学号或姓名信息。

然后，我们将通过 `mov` 指令将该字符串逐个字符输出到控制台或显示器。具体的输出方式取决于内核的输出机制，通常会通过端口或内存地址直接操作硬件。

## ● 实验步骤：

### 1. 文件结构



### 2. 运行并查看是否加载

我们使用如下语句编译。

```
make
```

在 `qemu` 的 `debug` 模式下加载运行。

```
make debug
```

我们在 `gdb` 下使用 `x/256gx 0x8880` 命令可以查看我们是否已经放入默认的中



断描述符，如下所示。

```
x/256gx 0x8880

linhongyu@linhongyu-VirtualBox: ~/桌面/lab4/lab4.6/build
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
r_kernel -Itext 0x00020000 --oformat binary
dd if=mbr.bin of=../run/hd.img bs=512 count=1 seek=0 conv=notrunc
记录了1+0 的读入
记录了1+0 的写出
512 bytes copied, 0.000137257 s, 3.7 MB/s
dd if=bootloader.bin of=../run/hd.img bs=512 count=5 seek=1 conv=notrunc
记录了0+1 的读入
记录了0+1 的写出
281 bytes copied, 8.6534e-05 s, 3.2 MB/s
dd if=kernel.bin of=../run/hd.img bs=512 count=145 seek=6 conv=notrunc
记录了145+0 的读入
记录了145+0 的写出
74240 bytes (74 kB, 72 KiB) copied, 0.000213167 s, 348 MB/s
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.6/build$ make debug
qemu-system-i386 -S -s -parallel stdio -hda ../run/hd.img -serial null&
WARNING: Image format was not specified for '../run/hd.img' and probing guessed
raw.

Automatically detecting the format is dangerous for raw images, write o
perations on block 0 will be restricted.
Specify the 'raw' format explicitly to remove the restrictions.
gnome-terminal -e "gdb -q -tui -x ../run/gdbinit"
# 参数“-e”弃用并可能在 gnome-terminal 的后续版本中移除。
# 使用“-- ”以结束选项并将要执行的命令行追加至其后。
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.6/build$

remote Thread 1 In:
0x8880: 0x0000000000000000 0x0000000000000000
0x8890: 0x0000000000000000 0x0000000000000000
0x88a0: 0x0000000000000000 0x0000000000000000
0x88b0: 0x0000000000000000 0x0000000000000000
0x88c0: 0x0000000000000000 0x0000000000000000
0x88d0: 0x0000000000000000 0x0000000000000000
0x88e0: 0x0000000000000000 0x0000000000000000
0x88f0: 0x0000000000000000 0x0000000000000000
0x8900: 0x0000000000000000 0x0000000000000000
0x8910: 0x0000000000000000 0x0000000000000000
0x8920: 0x0000000000000000 0x0000000000000000
---Type <return> to continue, or q <return> to quit---
```

### 3. 编译运行可见中断触发

```
make && make run
```

修改函数自己的中断处理函数，输出学号内容

```
ASM_UNHANDLED_INTERRUPT_INFO db '23320093LHY'

                                db 0

ASM_IDTR dw 0

                                dd 0

; void asm_unhandled_interrupt()
asm_unhandled_interrupt:

    cli

    mov esi, ASM_UNHANDLED_INTERRUPT_INFO

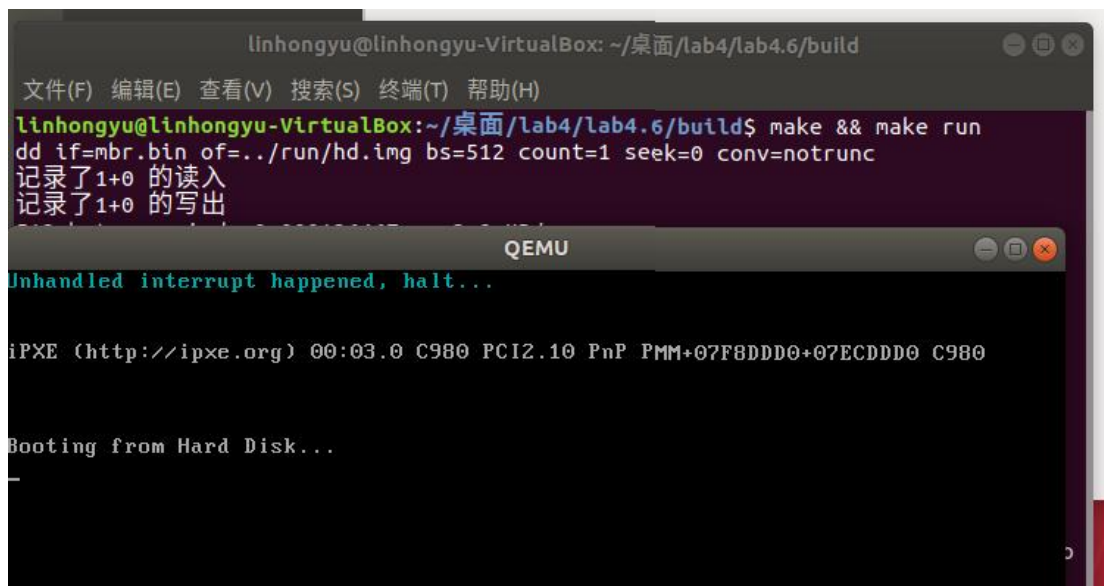
    xor ebx, ebx

mov ah, 0x03
```

- 实验结果展示：通过执行前述代码，可得下图结果。

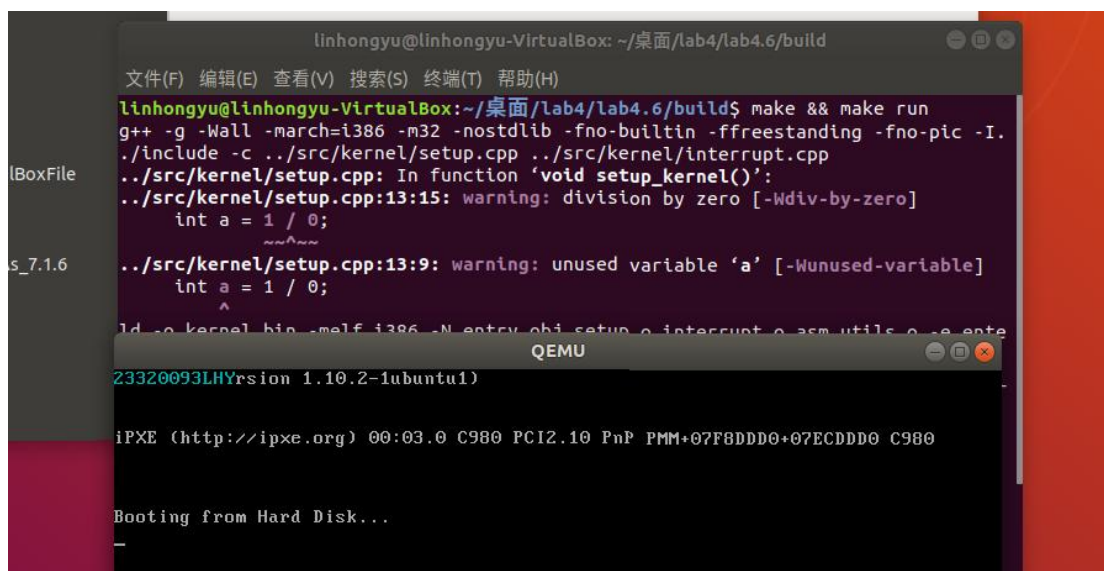
#### 1. 中断触发





```
linhongyu@linhongyu-VirtualBox: ~/桌面/lab4/lab4.6/build
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.6/build$ make && make run
dd if=mbr.bin of=../run/hd.img bs=512 count=1 seek=0 conv=notrunc
记录了1+0 的读入
记录了1+0 的写出
QEMU
Unhandled interrupt happened, halt...
iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DD0+07ECDD0 C980
Booting from Hard Disk...
```

## 2. 输出学号



```
linhongyu@linhongyu-VirtualBox: ~/桌面/lab4/lab4.6/build
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.6/build$ make && make run
g++ -g -Wall -march=i386 -m32 -nostdlib -fno-builtin -ffreestanding -fno-pic -I.
./include -c ./src/kernel/setup.cpp ./src/kernel/interrupt.cpp
./src/kernel/setup.cpp: In function 'void setup_kernel()':
./src/kernel/setup.cpp:13:15: warning: division by zero [-Wdiv-by-zero]
    int a = 1 / 0;
               ^
./src/kernel/setup.cpp:13:9: warning: unused variable 'a' [-Wunused-variable]
    int a = 1 / 0;
    ^
ld -o kernel.bin -melf-i386 -N entry.o hi setup.o interrupt.o asm_utils.o -e entry
23320093LHYrsion 1.10.2-1ubuntu1)
iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DD0+07ECDD0 C980
Booting from Hard Disk...
```

## 实验任务 4

- 任务要求：时钟中断的处理。

复现网址中“8259A 编程——实时钟中断的处理”部分，要求：仿照该章节中使用C 语言来实现时钟中断的例子，利用 C/C++ 、 InterruptManager 、 STDIO 和你自己封装的类来实现你的时钟中断处理过程(例如，通过时钟中断，你可以在屏幕的第一行实现一个跑马灯。跑马灯显示自己学号和英文名，即类似于 LED 屏幕显示的效果)，保存结果截图并说说你的思路 and 做法。

注意：不要使用纯汇编的方式来实现。

- 思路分析：

时钟中断的触发与处理：

时钟中断是通过定时器中断（8259A 定时器）触发的，每次定时器中断发生时，CPU 会跳转到指定的中断处理程序。

需要通过 `asm_time_interrupt_handler` 汇编代码来捕获中断，并调用 C++ 的 `c_time_interrupt_handler` 函数来处理每次中断。

屏幕输出：

由于我们需要在屏幕的第一行实现跑马灯效果，输出需要通过 `STDIO` 类来进行。该类封装了屏幕输出的细节，包括打印字符和管理光标位置。

跑马灯效果的实现依赖于循环输出字符，并通过时间的推移（中断次数）决定字符的颜色或位置。

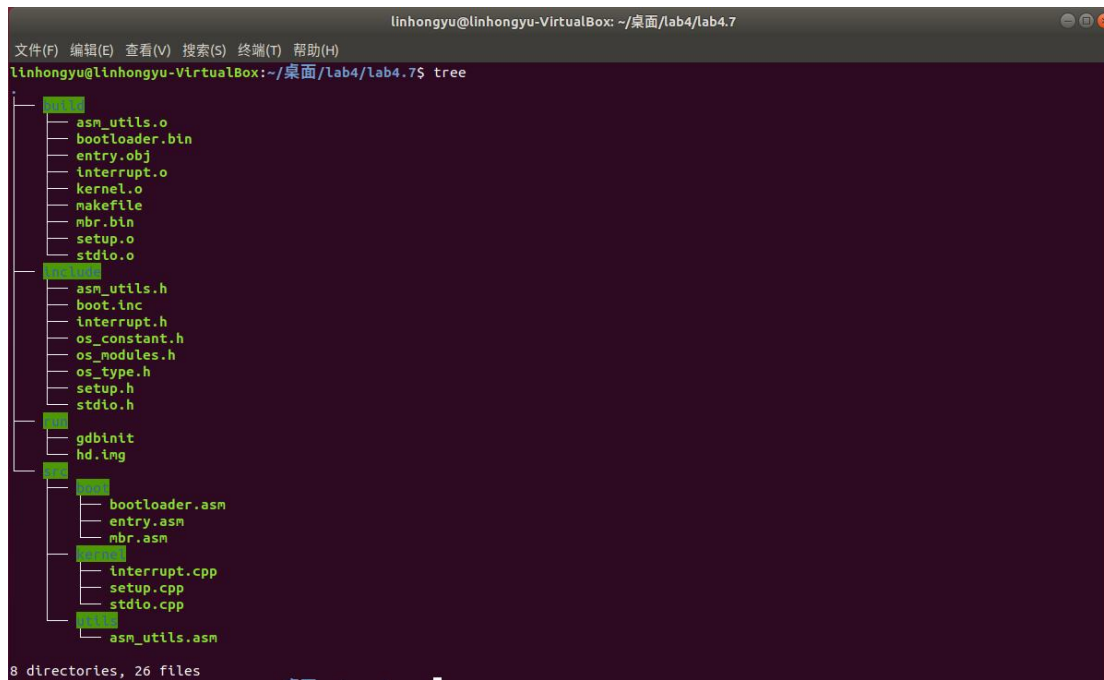
跑马灯的实现：

每次时钟中断触发时，`times` 变量会递增，这表示时钟中断发生的次数。通过 `times % 36` 的方式，控制字符在屏幕上的位置或颜色，模拟跑马灯的效果。

字符串 "23320093\_LinHongYu" 将逐字符打印，利用 `i - times % 36 == 1` 的条件控制跑马灯效果中的字符颜色。

- 实验步骤：

1. 总体目录结构



```
linhongyu@linhongyu-VirtualBox: ~/桌面/lab4/lab4.7
linhongyu@linhongyu-VirtualBox:~/桌面/lab4/lab4.7$ tree
.
├── asm
│   ├── asm_utils.o
│   ├── bootloader.bin
│   ├── entry.obj
│   ├── interrupt.o
│   ├── kernel.o
│   ├── makefile
│   ├── mbr.bin
│   ├── setup.o
│   └── stdio.o
├── include
│   ├── asm_utils.h
│   ├── boot.inc
│   ├── interrupt.h
│   ├── os_constant.h
│   ├── os_modules.h
│   ├── os_type.h
│   ├── setup.h
│   └── stdio.h
├── src
│   ├── gdbinit
│   └── hd.img
├── utils
│   ├── bootloader.asm
│   ├── entry.asm
│   └── mbr.asm
└── kernel
    ├── interrupt.cpp
    ├── setup.cpp
    ├── stdio.cpp
    └── asm_utils.asm

8 directories, 26 files
```

代码整体结构分析，实验项目实现了简单的操作系统内核，包括屏幕输出、中断处理等功能。以下是代码的整体结构和模块之间的调用关系：

### 1.1 主要模块

引导程序（Bootloader）：

src/boot/mbr.asm：主引导记录，负责加载 bootloader

src/boot/bootloader.asm：引导加载程序，负责设置 GDT、切换到保护模式、加载内核

### 1.2 内核初始化：

src/kernel/setup.cpp：内核设置函数，初始化各个管理器

### 1.3 中断管理：

src/kernel/interrupt.cpp：中断管理器实现，包含跑马灯效果的代码

include/interrupt.h：中断管理器类的定义

### 1.4 屏幕输出：

src/kernel/stdio.cpp：屏幕输出的实现

include/stdio.h：STDIO 类的定义

### 1.5 汇编工具：

src/utils/asm\_utils.asm：汇编实现的各种工具函数，包括中断处理

## 2. 运行

```
Make
Make run
```

模块之间的调用关系：

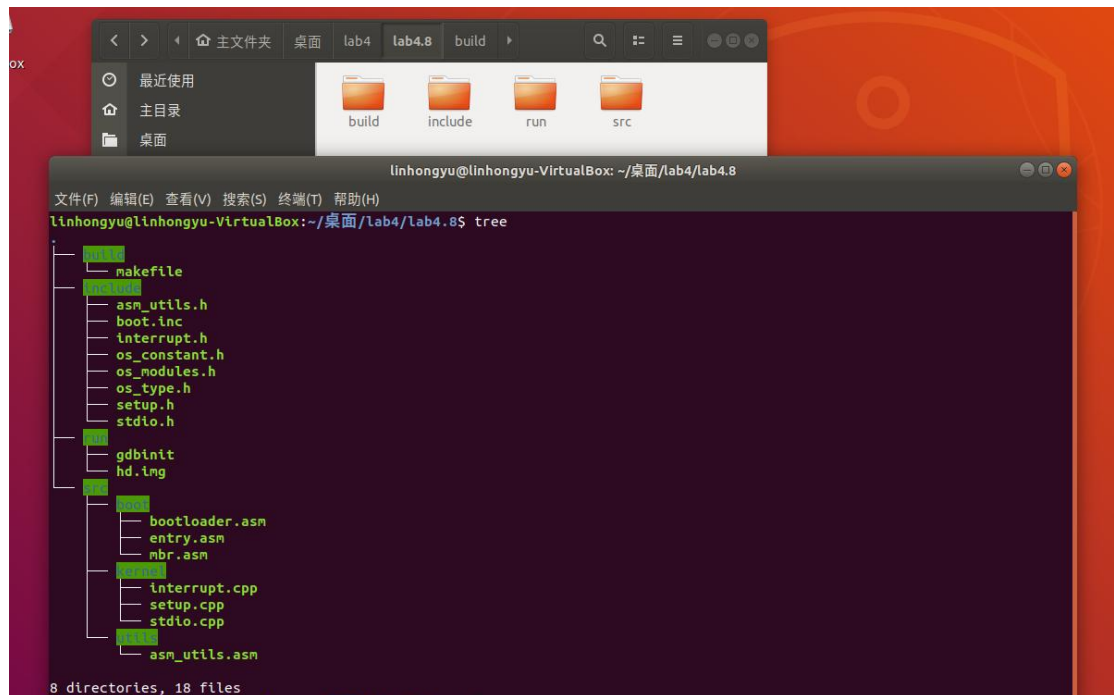
2.1 首先，mbr.asm 被 BIOS 加载并执行，它加载 bootloader.asm  
bootloader.asm 设置 GDT，切换到保护模式，然后加载内核

2.2 内核从 setup\_kernel 函数开始执行：

这里初始化了 interruptManager 和 stdio，然后启用了时钟中断。当时钟中断触发时，会调用 asm\_time\_interrupt\_handler（在 asm\_utils.asm 中定义），它又会调用 C++ 实现的 c\_time\_interrupt\_handler 函数

3. 改造自己的项目，输出跑马灯效果

3.1 项目结构



3.2 跑马灯设计

跑马灯在 c\_time\_interrupt\_handler 函数中实现，

位于 src/kernel/interrupt.cpp

实现过程如下：

3.2.1 清空屏幕：每次中断时，先通过循环清空屏幕，确保没有旧的字符干扰。

3.2.2 递增 times: times 变量记录中断发生的次数，每次中断都会递增，影响字符在屏幕上的显示位置。

3.2.3 输出字符串：字符串 str 被逐字符打印在屏幕上。

3.2.4 跑马灯效果：通过  $i - \text{times} \% 36 == 1$  这一条件，根据 `times` 的值决定哪个字符使用特殊颜色，从而模拟跑马灯的移动效果

#### 4. 中断处理函数

```
//中断处理函数
extern "C" void c_time_interrupt_handler()
{
    //清空屏幕
    for (int i = 0; i < 80; ++i)
    {
        stdio.print(0, i, ' ', 0x07);
    }

    //中断发生的次数++
    ++times;

    //输出字符串
    char str[] = "23320093_LinHongYu";
    //移动光标到(0,0)输出字符
    stdio.moveCursor(0);

    //跑马灯
    for(int i = 0; str[i]; ++i )
    {
        if(i - times % 36 == 1)
            stdio.print(str[i], 0x03); //青色显示
        else
            stdio.print(str[i]); //默认颜色
    }
}
```

#### 5. 调用 `stdio` 类中的输出部分

```
void STDIO::print(uint x, uint y, uint8 c, uint8 color)
{
    if (x >= 25 || y >= 80)
    {
        return;
    }

    uint pos = x * 80 + y;
    screen[2 * pos] = c;
    screen[2 * pos + 1] = color;
}

void STDIO::print(uint8 c, uint8 color)
{
    uint cursor = getCursor();
```

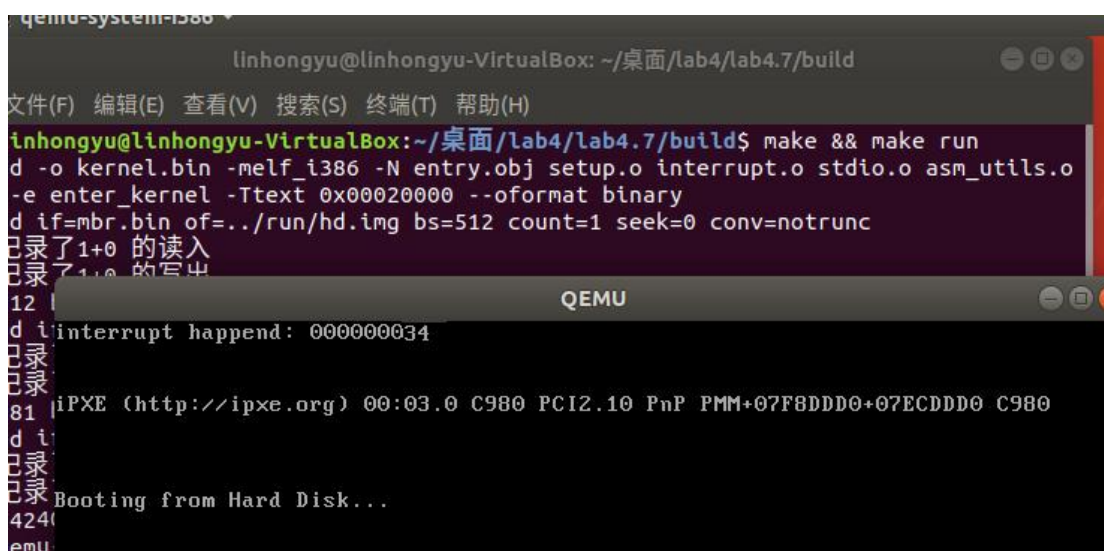
```

screen[2 * cursor] = c;
screen[2 * cursor + 1] = color;
cursor++;
if (cursor == 25 * 80)
{
    rollUp();
    cursor = 24 * 80;
}
moveCursor(cursor);
}
void STDIO::print(uint8 c)
{
    print(c, 0x07);
}

```

- 实验结果展示：通过执行前述代码，可得下图结果。

### 1. 复现结果

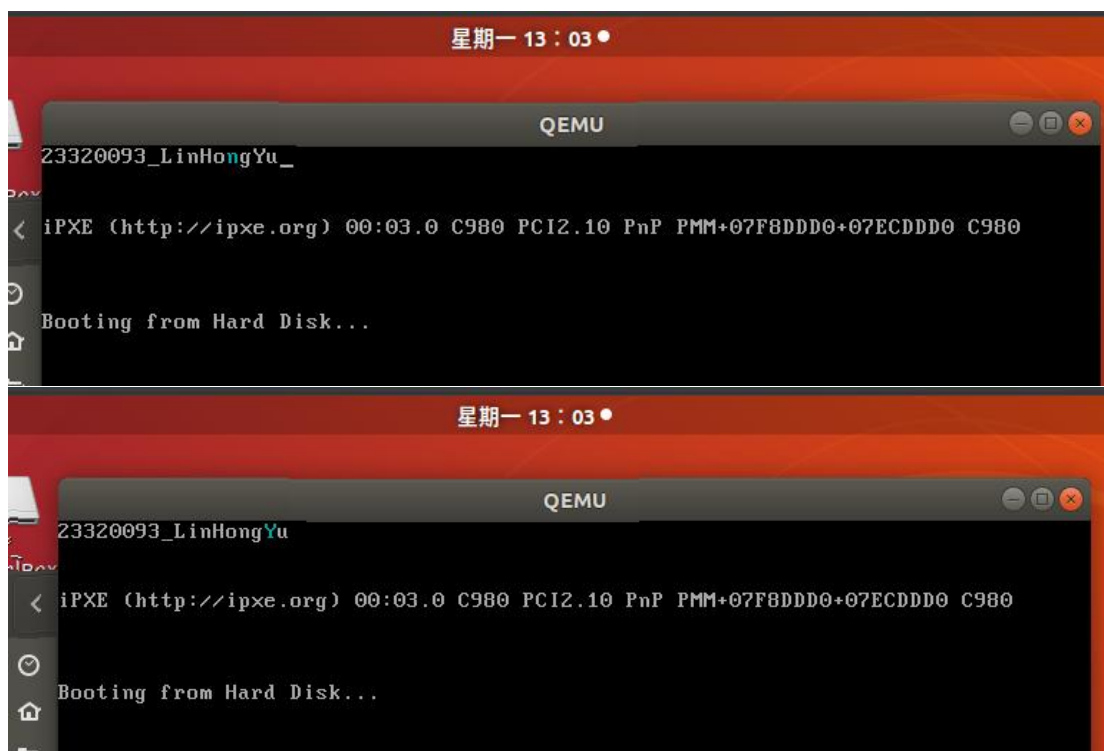


### 2. 跑马灯结果

分几次进行截图，可观察到呈现出跑马灯的效果







## Section 5 实验总结与心得体会

1.这几个实验报告涉及了内核编程、汇编与 C/C++混合编程、中断处理、以及硬件级编程等多个方面，是对计算机系统底层原理的深刻探索。从这些实验中，我不仅学到了如何操作系统内核的构建，还理解了如何在实际应用中处理硬件中断、实现屏幕输出等关键功能。

2.通过这些实验，我不仅加深了对计算机系统底层原理的理解，还培养了在面对复杂问题时的解决能力。操作系统内核的构建并不是一蹴而就的，它需要我们掌握很多硬件原理和软件实现技巧。在实际编程过程中，汇编语言和 C/C++混合编程、中断管理、内存管理等技术都发挥了重要作用，这些知识将在我以后的学习和开发中继续应用。

## Section 6 对实验的改进建议和意见

发现了说明书上的两处错误，不过只是命令的文件名错误，如下

我们生成 `main.c` 代码对应的汇编代码。命令如下，其中，`-masm=intel` 是为了生成intel风格的汇编代码，否则默认AT&T风格的代码。

```
gcc -o hello.s -S hello.c -masm=intel
```

生成的 `hello.s` 内容如下。

## Section 7 附录：参考资料清单

1. <https://blog.csdn.net/ZBraveHeart/article/details/123187908>
2. [https://blog.csdn.net/Sugar\\_wolf/article/details/134313363?fromshare=blogdetail&sharetype=blogdetail&sharerId=134313363&sharerefer=PC&sharesource=Full\\_cpp&sharefrom=from\\_link](https://blog.csdn.net/Sugar_wolf/article/details/134313363?fromshare=blogdetail&sharetype=blogdetail&sharerId=134313363&sharerefer=PC&sharesource=Full_cpp&sharefrom=from_link)
3. [https://blog.csdn.net/xiaofeixia002X/article/details/136053686?fromshare=blogdetail&sharetype=blogdetail&sharerId=136053686&sharerefer=PC&sharesource=Full\\_cpp&sharefrom=from\\_link](https://blog.csdn.net/xiaofeixia002X/article/details/136053686?fromshare=blogdetail&sharetype=blogdetail&sharerId=136053686&sharerefer=PC&sharesource=Full_cpp&sharefrom=from_link)

## Section 8 附录：代码清单

代码见压缩包文件